Unit II

2. Introduction to Digital Image Representation

2.1 Elements of Digital Image Processing System
2.2 Digital Image Representation
2.3 Reading, displaying and writing images
2.4 Data classes and Image types
2.5 Converting between data classes and image types
2.6 Introduction to M-function Programming

2.1 Elements of Digital Image Processing System:

A **Digital Image Processing (DIP) System** consists of several components that work together to capture, process, analyze, and display digital images. These elements help perform tasks such as enhancement, restoration, compression, and interpretation of images.

Key Elements of a Digital Image Processing System:

1. **Image Acquisition:**
   This is the first step where an image is captured using devices like cameras, scanners, or sensors. It includes converting optical images into digital form using sampling and quantization.

2. **Image Storage:**
   After acquisition, digital images are stored in memory devices such as RAM, hard disks, cloud storage, or specialized image databases. Efficient storage techniques may include compression.

3. **Image Processing Hardware:**
   This includes the processors (like GPUs, DSPs, microprocessors) used to perform operations such as filtering, enhancement, resizing, and transformations. Hardware accelerates processing speed.

4. **Image Processing Software:**
   Software provides tools and algorithms for tasks like image enhancement, segmentation, edge detection, noise removal, and pattern recognition. Examples include MATLAB, OpenCV, and Python libraries.

5. **Display and Visualization:**
   Images are displayed on monitors, screens, or printers for interpretation. High-resolution displays allow users to analyze fine details and processed results.

6. **Image Analysis and Interpretation:**
   In this stage, meaningful information is extracted from the image. It includes object detection, measurement, classification, and decision-making (used in medical imaging, biometrics, remote sensing, etc.).

These elements together form a complete digital image processing system that captures, processes, analyzes, and displays images for various scientific, industrial, and medical applications.

2.2 Digital Image Representation:

Digital Image Representation refers to the method of storing, displaying, and processing images in a computer system. A digital image is represented as a two-dimensional array of small picture elements called

pixels, where each pixel holds a numerical value that represents the intensity or color at that point.

Key Points about Digital Image Representation:

1. **Pixel (Picture Element):**
   A digital image is made up of tiny dots called pixels. Each pixel contains an intensity value (for grayscale) or color value (for color images). Higher pixel count gives better image quality.

2. **Resolution:**
   It refers to the number of rows and columns of pixels in an image (e.g., 1920×1080). Higher resolution means more pixels and clearer, more detailed images.

3. **Intensity Levels / Gray Levels:**
   Grayscale images represent intensity values using a range of gray shades. Most digital images use 8-bit representation, giving **256 gray levels** (0 = black, 255 = white).

4. **Color Image Representation:**
   Color images are usually represented using the **RGB model**, where each pixel has three components:
   R (Red)
   G (Green)
   B (Blue)
   Each component typically uses 8 bits, making a total of **24 bits per pixel**.

5. **Bit Depth:**
   Bit depth defines how many bits are used to represent each pixel. Higher bit depth allows more color or intensity variations.
   Examples:
   1-bit = 2 levels (black/white)
   8-bit = 256 gray levels
   24-bit = 16 million colors

Digital Image Representation allows computers to store, display, and manipulate images efficiently for various applications like medical imaging, remote sensing, photography, and machine vision.

2.3 Reading, displaying and writing images:

1. Reading Images:

Reading an image means importing a digital image from a file into the computer's memory so that it can be processed using image processing techniques. The process converts the file format (such as JPG, PNG, BMP) into a numerical matrix of pixel values.

Key Points:

1. **Definition:**
   Reading an image refers to loading an image file and converting it into a matrix where each element represents a pixel.

2. **Function Used (in MATLAB/Python):**
   MATLAB: `imread('image.jpg')`
   Python OpenCV: `cv2.imread('image.jpg')`

3. **Matrix Representation:**
   A grayscale image becomes a 2D matrix of intensity values.
   A color image becomes a 3D matrix (rows × columns × 3 channels).

4. **Importance:**
   Reading is the first step in any digital image processing task such as enhancement, filtering, or segmentation.

5. **Supports Multiple Formats:**
   The system can read several formats like JPG, PNG, BMP, TIFF, etc., making it flexible for various applications.

2. Displaying Images:

Displaying an image means showing the image on a screen or window so that the user can view it. This helps in visualizing the results of processing techniques like enhancement, noise removal, or filtering.

Key Points:

1. **Definition:**
   Displaying an image is the process of presenting a digital image visually on a monitor or GUI window.

2. **Function Used:**
   MATLAB: `imshow(image)`
   Python OpenCV: `cv2.imshow("Title", image)`

3. **Visualization of Pixel Data:**
   The pixel matrix is converted into a visual form where intensity or color values become viewable images.

4. **Used for Analysis:**
   Displaying is essential for checking image quality, verifying results, and comparing before-and-after processing outputs.

5. **Multiple Display Options:**
   Systems support zooming, adjusting colormaps, showing histograms, and displaying multiple images in subplots.

3. Writing Images:

Writing an image refers to saving the processed image back to a file. It stores the modified pixel matrix as an image in a chosen format.

Key Points:

1. **Definition:**
   Writing an image means exporting the pixel matrix to an external file, such as JPG, PNG, BMP, or TIFF.

2. **Function Used:**
   MATLAB: `imwrite(image, 'output.jpg')`
   Python OpenCV: `cv2.imwrite('output.jpg', image)`

3. **Preserves Processing Results:**
   After performing enhancement, filtering, segmentation, etc., the final result is saved for future use.

4. **Supports Compression:**
   Writing allows saving images in different formats that may use compression (like JPG) or lossless storage (like PNG/TIFF).

5. **Applications:**
   Writing is used in medical imaging, document scanning, graphics design, and machine learning datasets where processed images must be stored.

2.4 Data classes and Image types:

1. Data Classes:

In Digital Image Processing, **data classes** refer to the types of numerical data used to store pixel values in an image. Different data classes determine the range, precision, and memory required for representing image pixels.

Key Points about Data Classes:

1. **Definition:**
   Data classes specify the format and size of pixel values stored in an image, such as integer or floating-point types.

2. **Common Data Classes:**

   uint8:
   8-bit unsigned integer (0–255). Most commonly used for grayscale and RGB images.

   uint16:
   16-bit unsigned integer (0–65,535). Used for high-precision images like medical and scientific data.

   int16:
   16-bit signed integer (−32,768 to 32,767). Used in specialized applications.

   single (float32) / double (float64):
   Floating-point classes used for calculations, filtering, normalization, or scientific processing.

3. **Memory Requirement:**
   Higher data classes (like 16-bit and 32-bit) require more memory but provide higher accuracy and dynamic range.

4. **Purpose:**
   Data classes help ensure correct processing results because many algorithms depend on pixel value range and precision.

5. **Usage:**
   Different data classes are used in medical imaging, satellite images, scientific imaging, and applications requiring high dynamic range.

Data classes are essential for selecting the correct representation of pixel values based on required accuracy and processing needs.

2. Image Types:

Image types describe the **structure, format, and characteristics** of images used in digital image processing. Different image types store and represent image data in different ways.

Key Types of Digital Images:

1. **Binary Image:**
   Each pixel has only two values: 0 and 1 (black or white). Used in document processing, thresholding, and shape analysis.

2. **Grayscale Image:**
   Pixels represent shades of gray with intensity values typically ranging from 0 to 255 (8 bit). Commonly used in medical and industrial imaging.

3. **RGB Color Image:**
   Each pixel contains three components: Red, Green, and Blue. Represented as a 3D array (rows × columns × 3). Used in natural photographs.

4. **Indexed Image:**
   Pixels store color indices instead of actual RGB values. A colormap (lookup table) maps each index to an RGB color. Used for reducing memory usage.

5. **Multispectral / Hyperspectral Image:**
   Contains multiple bands beyond visible colors, captured by sensors. Used in remote sensing, agriculture, and scientific imaging.

Importance:

Different image types are used depending on storage needs, processing requirements, and application domains like medical imaging, biometrics, satellite imaging, and computer vision.

2.5 Converting between data classes and image types:

1. Converting Between Data Classes:

In Digital Image Processing, images may need to be converted from one **data class** to another depending on the requirements of algorithms, memory usage, or precision. Converting data classes changes the pixel value range and storage format without altering the actual visual content of the image.

Key Points:

1. **Definition:**
   Conversion between data classes refers to changing the numerical representation of pixel values from one format (e.g., uint8) to another (e.g., double or uint16).

2. **Common Conversions:**
   `uint8` ↔ `double`
   `uint8` ↔ `uint16`
   `double` ↔ `single`
   These conversions help match image data with specific processing functions.

3. **Function Used:**
   In MATLAB: `im2double()`, `im2uint8()`, `im2uint16()`, `im2single()`

4. **Purpose:**
   Algorithm compatibility: Many algorithms require images in double precision.
   Memory management: Converting to `uint8` reduces file size.
   Precision control: Scientific or medical imaging uses `uint16` for high dynamic range.

5. **Range Adjustment:**
   When converting, pixel values may be scaled:
   `uint8 (0–255)` → `double (0–1)`

`uint16 (0–65535)` → `uint8 (0–255)`
Proper scaling ensures accurate brightness and contrast.

Converting between data classes allows efficient processing, accurate results, and compatibility with different algorithms.

2. Converting Between Image Types:

Digital images can exist in different **image types** such as binary, grayscale, RGB, and indexed images. Converting between image types helps in applying appropriate processing operations or visualizing images in different forms.

Key Points:

1. **Definition:**
   Conversion between image types means changing the structure and representation of an image, such as converting an RGB image to grayscale or a grayscale image to a binary image.

2. **Common Conversions:**
   RGB → Grayscale: Converts 3-channel color images to single-channel intensity images.
   Grayscale → Binary: Uses thresholding to convert intensity values to 0 or 1.
   Indexed → RGB: Applies a colormap to convert index values into actual RGB colors.
   RGB → Indexed: Reduces the number of colors for memory-efficient storage.

3. **Functions Used (MATLAB):**
   `rgb2gray()`
   `im2bw()` or `imbinarize()`
   `ind2rgb()`
   `rgb2ind()`

4. **Purpose:**
   Simplify processing tasks (e.g., edge detection works better on grayscale than RGB).
   Improve storage efficiency (indexed images use less memory).
   Prepare images for segmentation, enhancement, or analysis.

5. **Applications:**
   Used in computer vision, medical imaging, OCR, machine learning, compression, and graphics where images must be converted to suitable formats before processing.

Conversion between image types ensures that images match the required format for specific algorithms and processing tasks.

2.6 Introduction to M-function Programming:

M-function programming refers to writing user-defined functions in MATLAB using files that have the extension **".m"**. These files contain a set of commands grouped together to perform a specific task. M functions help modularize code, improve reusability, and make programs easier to understand and maintain.

Key Points:

1. **Definition:**
   An M-function is a MATLAB program written in a separate `.m` file that starts with the keyword `function`. It accepts input arguments, processes them, and returns output values.

2. **Structure of an M-Function:**

Every M-function includes:

A **function definition line**

Input arguments

Output arguments

A **body** containing MATLAB statements

Example:

`function y = square(x)`

3. **Separation from Script Files:**

Unlike script files that run sequential commands, M-functions are reusable blocks of code that can be called repeatedly with different inputs from the Command Window or other programs.

4. **Advantages:**

Promotes modular programming

Reduces code repetition

Easy to debug and maintain

Allows building complex applications using smaller functions

5. **Types of M-Files:**

MATLAB supports:

Function M-files (user-defined functions)

Script M-files (sequences of commands)

M-functions are essential for creating custom operations and advanced programs.

M-function programming enables structured, efficient, and reusable coding in MATLAB, making it an important feature for scientific and engineering applications.